UNISENAI
CURSO SUPERIOR DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS (EAD)

JOÃO VÍTOR MARTINS DA SILVA
MARIO NOBRE PORTO

AV 03 - CONSTRUÇÃO DO MVP (PARTE 1)
AUTOMAÇÃO DA EMISSÃO DE CERTIFICADOS DE CALIBRAÇÃO

FLORIANÓPOLIS
2026

JOÃO VÍTOR MARTINS DA SILVA
MARIO NOBRE PORTO

AV 03 - CONSTRUÇÃO DO MVP (PARTE 1)
AUTOMAÇÃO DA EMISSÃO DE CERTIFICADOS DE CALIBRAÇÃO

Trabalho apresentado à unidade curricular de Projeto Aplicado do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas (EAD) do UniSENAI, como requisito parcial para obtenção de nota.

Professor: Hugo Menezes Barra

FLORIANÓPOLIS
2026

1. Visão Geral

Nas Entregas 01 e 02 mapeamos o processo manual de emissão de certificados de calibração, que hoje depende de planilhas Excel descentralizadas, e modelamos o banco relacional SQLite schulz_calibracao.db, com as tabelas tecnico, equipamento e medicao.

A linguagem escolhida foi Python 3, com a biblioteca padrão sqlite3 e sem dependências externas.

Nesta entrega o modelo conceitual virou código. O MVP Parte 1 reúne:

a) as classes de domínio Tecnico, Equipamento e Medicao, com encapsulamento e validação;

b) a regra metrológica que decide automaticamente entre APROVADO e REPROVADO;

c) a classe de conexão e persistência BancoDados, que executa INSERT, SELECT, UPDATE e DELETE no SQLite;

d) um script de demonstração que cria o banco, insere dados de teste, executa consultas com JOIN e imprime o relatório de calibração no terminal.

1.1 Estrutura de arquivos

Arquivo	Papel
04_classes.py	Exceções, classes de domínio e camada de persistência
05_main.py	Script principal de demonstração (executável)
06_mvp_parte1.md	Este documento

1.2 Como executar

```
python 05_main.py
```

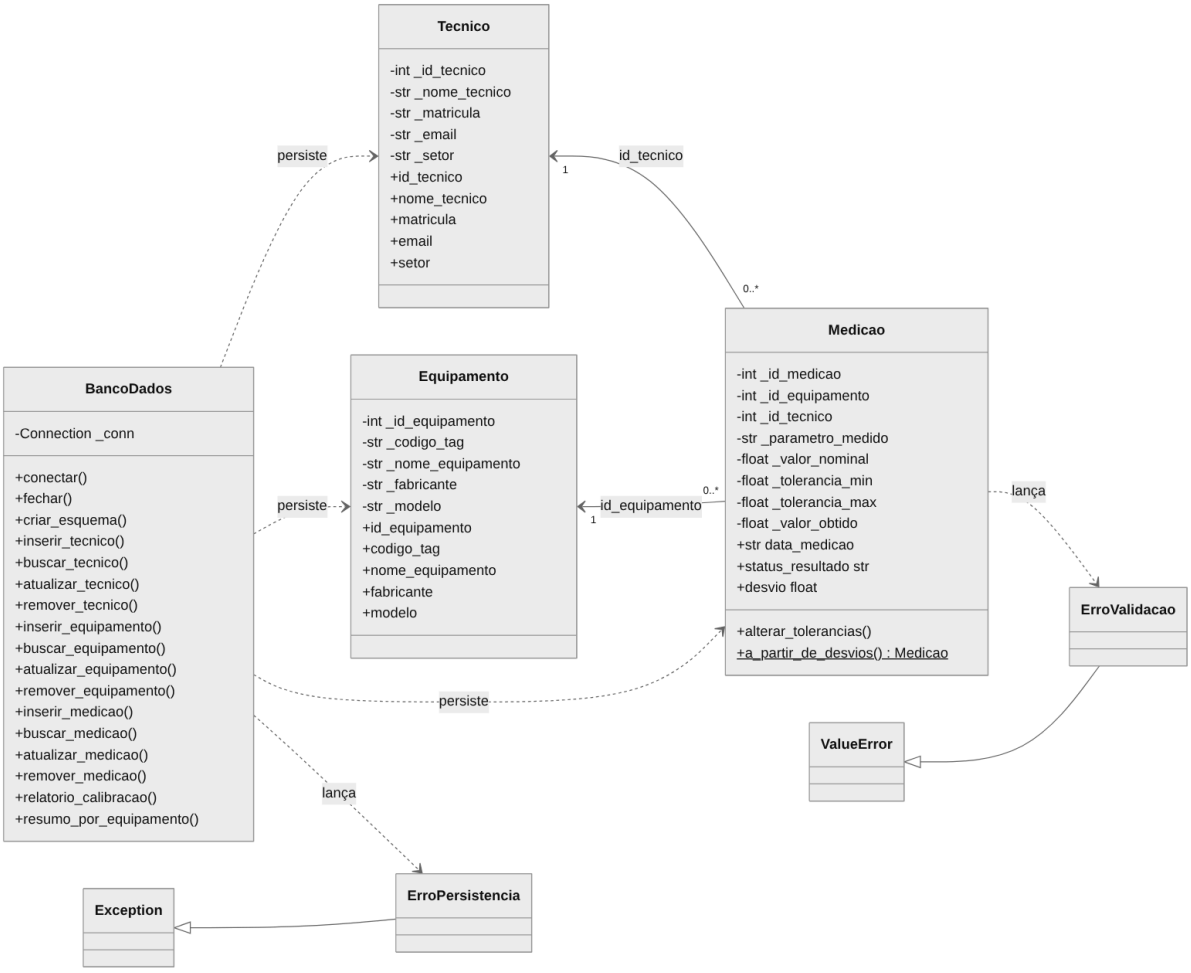
Basta Python 3.9 ou superior. O script recria o arquivo mvp_schulz_calibracao.db a cada execução, então o resultado é sempre reproduzível. Como 04_classes.py começa com um dígito, o 05_main.py o carrega com importlib.import_module("04_classes"), já que um import comum não aceita esse nome.

1.3 Alinhamento com a Entrega 02

O enunciado cita as tabelas de forma genérica (tecnicos, medicoes). O código segue o DDL real da Entrega 02 (03_banco.sql): tabelas no singular e colunas nome_tecnico, email, nome_equipamento, modelo, parametro_medido, tolerancia_min, tolerancia_max e status_resultado. Em relação aos nomes citados

no enunciado, nome corresponde a nome_tecnico e descricao corresponde a nome_equipamento. Assim, o banco criado pelo MVP é idêntico ao já entregue.

2. Arquitetura e Diagrama de Classes (UML textual)



Relações entre as classes:

a) Um Tecnico pode ter várias Medicao, e cada Medicao pertence a um único Tecnico (chave id_tecnico).

b) Um Equipamento também pode ter várias Medicao, e cada Medicao pertence a um único Equipamento (chave id_equipamento).

c) BancoDados depende das três classes de domínio: grava seus dados e reconstrói objetos a partir das linhas lidas.

A Medicao guarda apenas os ids do técnico e do equipamento, como nas chaves estrangeiras do banco, em vez de carregar os objetos completos. Quando o relatório precisa dos nomes, a junção é feita no próprio banco, por JOIN.

3. Padrões de POO Utilizados (Python)

Padrão ou princípio	Onde aparece	Motivo
Encapsulamento	Atributos privados (_matricula, _valor_obtido) expostos por @property, com setters que validam	Toda atribuição passa pela validação, então o objeto nunca fica em estado inválido
Atributo calculado	Medicao.status_resultado é uma @property sem setter	O status não pode ser digitado: sai sempre dos limites e do valor obtido. Isso resolve o risco, apontado na Entrega 02, de digitar por cima da fórmula na planilha
Imutabilidade parcial	Os id_* só aceitam uma atribuição, e os limites só mudam por alterar_tolerancias()	Protege a identidade do registro e garante que nominal e limites sejam validados juntos
Factory Method	Medicao.a_partir_de_desvios() (@classmethod)	Converte a notação de desenho (nominal 10,000 com desvios -0,020 e +0,030) nos limites absolutos gravados no banco
Separação de responsabilidades	Domínio (regras) e BancoDados (SQL)	O domínio não conhece SQL, e a persistência não conhece regras metrológicas
Gerenciador de contexto	with BancoDados(...) as bd: (__enter__ e __exit__)	Fecha a conexão mesmo quando ocorre um erro
Transação por operação	_transacao() usa with conn:	Faz commit em caso de sucesso e rollback automático em caso de exceção
Hierarquia de exceções própria	ErroValidacao e ErroPersistencia	Quem chama distingue dado inválido de falha de banco
DRY	_texto() e _numero()	Uma só implementação de validação, usada pelas três entidades

3.1 Regra de negócio metrológica

Definida no dicionário de dados da Entrega 02 e implementada em Medicao.status_resultado:

```
tolerancia_min = valor_nominal - |tolerância negativa|     (limite inferior)
tolerancia_max = valor_nominal + |tolerância positiva|     (limite superior)

APROVADO   se tolerancia_min <= valor_obtido <= tolerancia_max
REPROVADO  caso contrário
```

Alguns cuidados que o código toma:

a) Os limites são inclusivos: uma leitura exatamente no limite é aprovada (caso 4 do relatório, com 10,030 e limite 10,030).

b) A comparação usa uma margem EPSILON = 1e-9. Em ponto flutuante, 10.0 + 0.1 não é exatamente 10.1, e sem a margem uma leitura de 10,1 seria reprovada

por engano.

c) Valores no padrão brasileiro, como "10,030" exportado de planilha, são aceitos e convertidos.

d) O construtor rejeita tolerancia_min maior que tolerancia_max e também nominal fora da faixa.

3.2 Validações de domínio

Classe	Campo	Regra
Tecnico	nome_tecnico	Texto de 3 a 120 caracteres, com espaços normalizados
Tecnico	matricula	3 a 20 caracteres alfanuméricos, convertida para maiúsculas
Tecnico	email	Formato x@y.z, convertido para minúsculas
Equipamento	codigo_tag	Formato de tag (ex.: 703.1208), em maiúsculas
Equipamento	demais campos	Texto obrigatório, não vazio
Medicao	chaves estrangeiras	Inteiros positivos
Medicao	valores numéricos	Números finitos (rejeita texto, None, NaN, inf e booleanos)

4. Tratamento de Erros SQL

A validação acontece em duas camadas. O domínio rejeita o dado antes de chegar ao banco, e as restrições do SQLite (UNIQUE, FOREIGN KEY, CHECK e NOT NULL) funcionam como segunda barreira, caso algum caminho contorne as classes.

4.1 Mecanismos

a) Consultas parametrizadas. Todo valor entra por ?, nunca por concatenação de texto, o que evita SQL Injection. Testamos com o filtro x' OR '1'='1, que devolve lista vazia.

b) Integridade referencial ativa. O SQLite ignora chaves estrangeiras por padrão, então PRAGMA foreign_keys = ON é executado em toda conexão (conectar()).

c) Transações. As escritas rodam em with conn:, e qualquer falha desfaz a operação inteira.

d) Tradução de exceções. sqlite3.IntegrityError e sqlite3.Error são capturadas em um único ponto (_transacao() e _consultar()) e relançadas como ErroPersistencia, com mensagem legível e a causa original preservada (raise ... from e).

e) Erros de uso. Operar com o banco desconectado, ou atualizar um objeto que ainda não tem id, gera um erro explícito.

4.2 Erros cobertos

Situação	Quem bloqueia	Resultado
E-mail em formato inválido	Domínio	ErroValidacao
Valor obtido não numérico ("abc")	Domínio	ErroValidacao
tolerancia_min maior que tolerancia_max	Domínio	ErroValidacao
Nominal fora da faixa de tolerância	Domínio	ErroValidacao
Filtro de status inválido no relatório	Domínio	ErroValidacao
Matrícula ou tag duplicada	SQLite (UNIQUE)	ErroPersistencia (registro duplicado)
Medição com equipamento ou técnico inexistente	SQLite (FOREIGN KEY)	ErroPersistencia (integridade referencial)
Excluir técnico ou equipamento que tem medições	SQLite (ON DELETE RESTRICT)	ErroPersistencia
Banco não conectado	Persistência	ErroPersistencia

Todos os casos, exceto o último, são exercitados na seção 5 da saída do 05_main.py (matrícula duplicada e tag duplicada são testadas separadamente, o que dá nove testes). O caso de banco não conectado foi conferido em teste à parte.

5. Validação do Fluxo Funcional do MVP

O 05_main.py percorre o ciclo completo de uso, e cada etapa foi executada e conferida.

Etapa	O que o script faz	Resultado verificado
1	Conecta e cria o esquema (criar_esquema)	Três tabelas criadas, idênticas ao 03_banco.sql
2	Instancia 3 técnicos, 4 equipamentos e 7 medições	Objetos criados, e os ids gerados pelo banco voltam para os objetos
3	Persiste com INSERT	14 registros gravados, com o status calculado pela regra da classe
4	UPDATE: corrige a planicidade de 0,035 para 0,015	O status passa de REPROVADO para APROVADO e é confirmado relendo do banco
5	Provoca 9 erros propositais	Os 9 foram bloqueados e nenhum registro inválido foi gravado
6.1	Relatório com INNER JOIN (medição, equipamento e técnico)	7 linhas com tag, parâmetro, limites, valor obtido, status e técnico
6.2 e 6.3	Filtros por REPROVADO e pela tag 703.1208	Retornam só os registros esperados

Etapa	O que o script faz	Resultado verificado
6.4	LEFT JOIN com GROUP BY	Totais por equipamento, incluindo o 810.0042, que ainda não tem medições

5.1 Casos de teste da regra de status

Nominal	Limites	Obtido	Esperado	Motivo
25,000	24,950 a 25,030	25,012	APROVADO	Dentro da faixa
0,000	0,000 a 0,020	0,035	REPROVADO	Acima do limite superior
50,000	49,950 a 50,050	49,949	REPROVADO	Abaixo do limite inferior
10,000	9,980 a 10,030	10,030	APROVADO	Exatamente no limite (inclusivo)
10,000	9,950 a 10,100	10,100	APROVADO	Limite calculado em ponto flutuante (10.0 + 0.1)

5.2 Relatório de calibração (trecho da saída real)

```
 ID | TAG       | PARÂMETRO         |  NOMINAL |  LIM.MIN |  LIM.MAX |   OBTIDO | STATUS    | TÉCNICO
-------------------------------------------------------------------------------------------------------
  6 | 500.011   | Diâmetro Interno  |   40.000 |   39.995 |   40.005 |   40.002 | APROVADO  | Ana Beatriz Souza
  7 | 500.011   | Diâmetro Interno  |   40.000 |   39.995 |   40.005 |   40.011 | REPROVADO | Carlos Eduardo Pereira
  1 | 703.1208  | Diâmetro          |   25.000 |   24.950 |   25.030 |   25.012 | APROVADO  | Carlos Eduardo Pereira
  2 | 703.1208  | Planicidade       |    0.000 |    0.000 |    0.020 |    0.015 | APROVADO  | Carlos Eduardo Pereira
  3 | 703.1208  | Posição           |   50.000 |   49.950 |   50.050 |   49.949 | REPROVADO | Ana Beatriz Souza
```

6. Limitações e Próximos Passos (MVP Parte 2)

a) Interface: o MVP roda pelo terminal e ainda não tem tela nem entrada interativa de dados.

b) Importação: a leitura automática do relatório exportado pela máquina de medir por coordenadas (Calypso) ainda não existe. A validação já aceita vírgula decimal, o que facilita essa etapa.

c) Certificado: gerar o certificado de calibração a partir do relatório é o objetivo final do projeto e fica para a Parte 2.

d) Persistência: BancoDados concentra o CRUD das três entidades, o que serve para o tamanho do MVP. Se o sistema crescer, o passo natural é separar em um repositório por entidade (padrão DAO).

e) Testes: a validação foi feita executando o roteiro e com testes de borda manuais. Testes automatizados com unittest ou pytest ficam como próximo passo.

f) Dados de teste: todos os nomes, e-mails e valores do 05_main.py são fictícios.

7. Arquivos Entregues e Repositório

Os três arquivos solicitados para esta entrega são 04_classes.py (classes de domínio e persistência), 05_main.py (script de demonstração) e 06_mvp_parte1.md (esta documentação). Todos estão no repositório público abaixo.

https://github.com/maartinnss/unisenai_desafio_PA

Anexo A: código-fonte de 04_classes.py

```
# ============================================================================
# UNIDADE CURRICULAR: PROJETO APLICADO I | SCHULZ S.A.
# AV 03 (MVP Parte 1) | Arquivo 04: classes de domínio e conexão SQL
#
# O arquivo tem três partes:
#   1. Exceções próprias: ErroValidacao e ErroPersistencia
#   2. Domínio (POO): Tecnico, Equipamento e Medicao
#   3. Persistência (SQLite): BancoDados, com conexão, CRUD e relatórios
#
# O esquema SQL é o mesmo do 03_banco.sql da Entrega 02.
# ============================================================================
from __future__ import annotations

import math
import re
import sqlite3
from contextlib import contextmanager
from typing import Iterator, Optional

STATUS_APROVADO = "APROVADO"
STATUS_REPROVADO = "REPROVADO"

# Margem para comparar floats. Sem ela, um limite calculado como 10.0 + 0.1
# (que em binário não é exatamente 10.1) reprovaria uma leitura de 10.1.
EPSILON = 1e-9

_REGEX_EMAIL = re.compile(r"^[^@\s]+@[^@\s]+\.[^@\s]+$")


# ============================================================================
# 1. EXCEÇÕES PRÓPRIAS
# ============================================================================
class ErroValidacao(ValueError):
    """Dado inválido detectado no domínio, antes de chegar ao banco."""


class ErroPersistencia(Exception):
    """Falha de acesso ao SQLite, traduzida para uma mensagem legível."""


# Funções de validação reaproveitadas pelas três entidades.
def _texto(valor, campo: str, minimo: int = 1, maximo: int = 120) -> str:
    if not isinstance(valor, str):
        raise ErroValidacao(f"'{campo}' deve ser um texto.")
    valor = " ".join(valor.split())  # tira espaços duplicados e das pontas
    if not (minimo <= len(valor) <= maximo):
        raise ErroValidacao(
            f"'{campo}' deve ter entre {minimo} e {maximo} caracteres."
        )
    return valor


def _numero(valor, campo: str) -> float:
    """Aceita int, float e texto com vírgula decimal (padrão das planilhas)."""
    if isinstance(valor, bool):
        raise ErroValidacao(f"'{campo}' deve ser numérico.")
    if isinstance(valor, str):
        valor = valor.strip().replace(",", ".")
    try:
        numero = float(valor)
    except (TypeError, ValueError):
        raise ErroValidacao(f"'{campo}' deve ser numérico (recebido: {valor!r}).")
    if math.isnan(numero) or math.isinf(numero):
        raise ErroValidacao(f"'{campo}' deve ser um número finito.")
    return numero


# ============================================================================
# 2. CLASSES DE DOMÍNIO
# ============================================================================
class Tecnico:
```

```
    """Profissional calibrador (tabela `tecnico`)."""

    def __init__(self, nome_tecnico: str, matricula: str, email: str,
                 setor: str, id_tecnico: Optional[int] = None):
        self._id_tecnico = id_tecnico
        self.nome_tecnico = nome_tecnico
        self.matricula = matricula
        self.email = email
        self.setor = setor

    # O id é gerado pelo banco e só pode ser definido uma vez.
    @property
    def id_tecnico(self) -> Optional[int]:
        return self._id_tecnico

    @id_tecnico.setter
    def id_tecnico(self, valor: int) -> None:
        if self._id_tecnico is not None:
            raise ErroValidacao("id_tecnico já definido e não pode ser alterado.")
        self._id_tecnico = valor

    @property
    def nome_tecnico(self) -> str:
        return self._nome_tecnico

    @nome_tecnico.setter
    def nome_tecnico(self, valor: str) -> None:
        self._nome_tecnico = _texto(valor, "nome_tecnico", minimo=3)

    @property
    def matricula(self) -> str:
        return self._matricula

    @matricula.setter
    def matricula(self, valor: str) -> None:
        valor = _texto(valor, "matricula", minimo=3, maximo=20).upper()
        if not valor.isalnum():
            raise ErroValidacao("'matricula' deve conter apenas letras e números.")
        self._matricula = valor

    @property
    def email(self) -> str:
        return self._email

    @email.setter
    def email(self, valor: str) -> None:
        valor = _texto(valor, "email", maximo=150).lower()
        if not _REGEX_EMAIL.match(valor):
            raise ErroValidacao(f"'email' inválido: {valor!r}.")
        self._email = valor

    @property
    def setor(self) -> str:
        return self._setor

    @setor.setter
    def setor(self, valor: str) -> None:
        self._setor = _texto(valor, "setor")

    def __repr__(self) -> str:
        return (f"Tecnico(id={self.id_tecnico}, matricula={self.matricula!r}, "
                f"nome={self.nome_tecnico!r}, setor={self.setor!r})")


class Equipamento:
    """Instrumento submetido a calibração (tabela `equipamento`)."""

    def __init__(self, codigo_tag: str, nome_equipamento: str, fabricante: str,
                 modelo: str, id_equipamento: Optional[int] = None):
        self._id_equipamento = id_equipamento
        self.codigo_tag = codigo_tag
        self.nome_equipamento = nome_equipamento
        self.fabricante = fabricante
        self.modelo = modelo
```

```
    @property
    def id_equipamento(self) -> Optional[int]:
        return self._id_equipamento

    @id_equipamento.setter
    def id_equipamento(self, valor: int) -> None:
        if self._id_equipamento is not None:
            raise ErroValidacao("id_equipamento já definido e não pode ser alterado.")
        self._id_equipamento = valor

    @property
    def codigo_tag(self) -> str:
        return self._codigo_tag

    @codigo_tag.setter
    def codigo_tag(self, valor: str) -> None:
        valor = _texto(valor, "codigo_tag", minimo=2, maximo=30).upper()
        # As tags da Schulz têm formato como 703.1208 ou 500.011.
        if not re.fullmatch(r"[A-Z0-9][A-Z0-9.\-_/]*", valor):
            raise ErroValidacao(f"'codigo_tag' inválido: {valor!r}.")
        self._codigo_tag = valor

    @property
    def nome_equipamento(self) -> str:
        return self._nome_equipamento

    @nome_equipamento.setter
    def nome_equipamento(self, valor: str) -> None:
        self._nome_equipamento = _texto(valor, "nome_equipamento")

    @property
    def fabricante(self) -> str:
        return self._fabricante

    @fabricante.setter
    def fabricante(self, valor: str) -> None:
        self._fabricante = _texto(valor, "fabricante")

    @property
    def modelo(self) -> str:
        return self._modelo

    @modelo.setter
    def modelo(self, valor: str) -> None:
        self._modelo = _texto(valor, "modelo")

    def __repr__(self) -> str:
        return (f"Equipamento(id={self.id_equipamento}, tag={self.codigo_tag!r}, "
                f"nome={self.nome_equipamento!r})")


class Medicao:
    """Ensaio de calibração (tabela `medicao`).

    Regra do dicionário de dados da Entrega 02:
        tolerancia_min = valor_nominal - |tolerância negativa|
        tolerancia_max = valor_nominal + |tolerância positiva|
        APROVADO  se tolerancia_min <= valor_obtido <= tolerancia_max
        REPROVADO caso contrário

    O status não é um atributo editável: é calculado a cada consulta a partir
    dos limites e do valor obtido, então nunca fica inconsistente com eles.
    """

    def __init__(self, id_equipamento: int, id_tecnico: int,
                 parametro_medido: str, valor_nominal, tolerancia_min,
                 tolerancia_max, valor_obtido,
                 id_medicao: Optional[int] = None,
                 data_medicao: Optional[str] = None):
        self._id_medicao = id_medicao
        self.id_equipamento = id_equipamento
        self.id_tecnico = id_tecnico
        self.parametro_medido = parametro_medido
```

```
    # Os limites dependem do nominal, por isso são validados em conjunto.
    self._definir_limites(valor_nominal, tolerancia_min, tolerancia_max)
    self.valor_obtido = valor_obtido
    self.data_medicao = data_medicao  # preenchida pelo banco (CURRENT_TIMESTAMP)

@classmethod
def a_partir_de_desvios(cls, id_equipamento: int, id_tecnico: int,
                        parametro_medido: str, valor_nominal,
                        desvio_negativo, desvio_positivo, valor_obtido,
                        **kwargs) -> "Medicao":
    """Cria a medição a partir da notação de desenho (ex.: 10,000 com
    desvios -0,020 e +0,030), convertendo para os limites absolutos
    gravados no banco."""
    nominal = _numero(valor_nominal, "valor_nominal")
    neg = abs(_numero(desvio_negativo, "desvio_negativo"))
    pos = abs(_numero(desvio_positivo, "desvio_positivo"))
    return cls(id_equipamento, id_tecnico, parametro_medido, nominal,
               nominal - neg, nominal + pos, valor_obtido, **kwargs)

@property
def id_medicao(self) -> Optional[int]:
    return self._id_medicao

@id_medicao.setter
def id_medicao(self, valor: int) -> None:
    if self._id_medicao is not None:
        raise ErroValidacao("id_medicao já definido e não pode ser alterado.")
    self._id_medicao = valor

@property
def id_equipamento(self) -> int:
    return self._id_equipamento

@id_equipamento.setter
def id_equipamento(self, valor: int) -> None:
    self._id_equipamento = self._validar_fk(valor, "id_equipamento")

@property
def id_tecnico(self) -> int:
    return self._id_tecnico

@id_tecnico.setter
def id_tecnico(self, valor: int) -> None:
    self._id_tecnico = self._validar_fk(valor, "id_tecnico")

@staticmethod
def _validar_fk(valor, campo: str) -> int:
    if isinstance(valor, bool) or not isinstance(valor, int) or valor <= 0:
        raise ErroValidacao(f"'{campo}' deve ser um inteiro positivo.")
    return valor

@property
def parametro_medido(self) -> str:
    return self._parametro_medido

@parametro_medido.setter
def parametro_medido(self, valor: str) -> None:
    self._parametro_medido = _texto(valor, "parametro_medido")

def _definir_limites(self, nominal, minimo, maximo) -> None:
    nominal = _numero(nominal, "valor_nominal")
    minimo = _numero(minimo, "tolerancia_min")
    maximo = _numero(maximo, "tolerancia_max")
    if minimo > maximo + EPSILON:
        raise ErroValidacao(
            "tolerancia_min não pode ser maior que tolerancia_max."
        )
    if not (minimo - EPSILON <= nominal <= maximo + EPSILON):
        raise ErroValidacao(
            "valor_nominal deve estar dentro da faixa "
            "[tolerancia_min, tolerancia_max]."
        )
    self._valor_nominal = nominal
    self._tolerancia_min = minimo
```

```
        self._tolerancia_max = maximo

    @property
    def valor_nominal(self) -> float:
        return self._valor_nominal

    @property
    def tolerancia_min(self) -> float:
        return self._tolerancia_min

    @property
    def tolerancia_max(self) -> float:
        return self._tolerancia_max

    def alterar_tolerancias(self, valor_nominal, tolerancia_min,
                            tolerancia_max) -> None:
        """Único caminho para mudar os limites, validando os três juntos."""
        self._definir_limites(valor_nominal, tolerancia_min, tolerancia_max)

    @property
    def valor_obtido(self) -> float:
        return self._valor_obtido

    @valor_obtido.setter
    def valor_obtido(self, valor) -> None:
        self._valor_obtido = _numero(valor, "valor_obtido")

    @property
    def status_resultado(self) -> str:
        """Parecer técnico automático: APROVADO ou REPROVADO."""
        dentro = (self._tolerancia_min - EPSILON
                  <= self._valor_obtido
                  <= self._tolerancia_max + EPSILON)
        return STATUS_APROVADO if dentro else STATUS_REPROVADO

    @property
    def desvio(self) -> float:
        """Valor obtido menos o nominal, com sinal."""
        return self._valor_obtido - self._valor_nominal

    def __repr__(self) -> str:
        return (f"Medicao(id={self.id_medicao}, eq={self.id_equipamento}, "
                f"tec={self.id_tecnico}, {self.parametro_medido!r}, "
                f"obtido={self.valor_obtido}, {self.status_resultado})")


# ============================================================================
# 3. CAMADA DE PERSISTÊNCIA
# ============================================================================
_DDL = """
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS tecnico (
    id_tecnico INTEGER PRIMARY KEY AUTOINCREMENT,
    nome_tecnico TEXT NOT NULL,
    matricula TEXT NOT NULL UNIQUE,
    email TEXT NOT NULL,
    setor TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS equipamento (
    id_equipamento INTEGER PRIMARY KEY AUTOINCREMENT,
    codigo_tag TEXT NOT NULL UNIQUE,
    nome_equipamento TEXT NOT NULL,
    fabricante TEXT NOT NULL,
    modelo TEXT NOT NULL
);

CREATE TABLE IF NOT EXISTS medicao (
    id_medicao INTEGER PRIMARY KEY AUTOINCREMENT,
    id_equipamento INTEGER NOT NULL,
    id_tecnico INTEGER NOT NULL,
    parametro_medido TEXT NOT NULL,
    valor_nominal REAL NOT NULL,
```

```
    tolerancia_min REAL NOT NULL,
    tolerancia_max REAL NOT NULL,
    valor_obtido REAL NOT NULL,
    status_resultado TEXT NOT NULL CHECK (status_resultado IN ('APROVADO', 'REPROVADO')),
    data_medicao DATETIME DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (id_equipamento) REFERENCES equipamento (id_equipamento)
        ON DELETE RESTRICT ON UPDATE CASCADE,
    FOREIGN KEY (id_tecnico) REFERENCES tecnico (id_tecnico)
        ON DELETE RESTRICT ON UPDATE CASCADE
);
"""


class BancoDados:
    """Gerencia a conexão `sqlite3` e executa todos os comandos SQL.

    Os valores sempre entram por parâmetros (`?`), nunca por concatenação de
    texto, o que evita SQL Injection. As escritas rodam em transação: ou
    gravam tudo, ou desfazem (rollback).
    """

    def __init__(self, caminho: str = "schulz_calibracao.db"):
        self.caminho = caminho
        self._conn: Optional[sqlite3.Connection] = None

    # Conexão
    def conectar(self) -> "BancoDados":
        try:
            self._conn = sqlite3.connect(self.caminho)
            self._conn.row_factory = sqlite3.Row
            # O SQLite não aplica chaves estrangeiras por padrão: é preciso
            # ligar o PRAGMA a cada nova conexão.
            self._conn.execute("PRAGMA foreign_keys = ON;")
        except sqlite3.Error as e:
            raise ErroPersistencia(f"Não foi possível abrir '{self.caminho}': {e}")
        return self

    def fechar(self) -> None:
        if self._conn is not None:
            self._conn.close()
            self._conn = None

    def __enter__(self) -> "BancoDados":
        return self.conectar()

    def __exit__(self, *_exc) -> None:
        self.fechar()

    @property
    def conexao(self) -> sqlite3.Connection:
        if self._conn is None:
            raise ErroPersistencia("Banco desconectado. Chame conectar() primeiro.")
        return self._conn

    def criar_esquema(self) -> None:
        try:
            self.conexao.executescript(_DDL)
            self.conexao.execute("PRAGMA foreign_keys = ON;")
        except sqlite3.Error as e:
            raise ErroPersistencia(f"Falha ao criar o esquema: {e}")

    # Execução centralizada, com tradução dos erros do SQLite
    @contextmanager
    def _transacao(self) -> Iterator[sqlite3.Connection]:
        conn = self.conexao
        try:
            with conn:  # commit se der certo, rollback se der erro
                yield conn
        except sqlite3.IntegrityError as e:
            raise ErroPersistencia(self._traduzir_integridade(str(e))) from e
        except sqlite3.Error as e:
            raise ErroPersistencia(f"Erro de banco de dados: {e}") from e

    @staticmethod
```

```
def _traduzir_integridade(msg: str) -> str:
    if "UNIQUE" in msg:
        campo = msg.rsplit(".", 1)[-1]
        return f"Registro duplicado: já existe um registro com este '{campo}'."
    if "FOREIGN KEY" in msg:
        return ("Violação de integridade referencial: o registro referenciado "
                "não existe ou ainda possui medições vinculadas.")
    if "CHECK" in msg:
        return "Valor rejeitado por uma restrição CHECK do banco."
    if "NOT NULL" in msg:
        return f"Campo obrigatório ausente ({msg})."
    return f"Violação de integridade: {msg}"

def _consultar(self, sql: str, params: tuple = ()) -> list[sqlite3.Row]:
    try:
        return self.conexao.execute(sql, params).fetchall()
    except sqlite3.Error as e:
        raise ErroPersistencia(f"Erro ao consultar: {e}") from e

# ============================================================================
# Técnico
# ============================================================================
@staticmethod
def _linha_para_tecnico(r: sqlite3.Row) -> Tecnico:
    return Tecnico(r["nome_tecnico"], r["matricula"], r["email"],
                   r["setor"], id_tecnico=r["id_tecnico"])

def inserir_tecnico(self, t: Tecnico) -> int:
    with self._transacao() as c:
        cur = c.execute(
            "INSERT INTO tecnico (nome_tecnico, matricula, email, setor) "
            "VALUES (?, ?, ?, ?)",
            (t.nome_tecnico, t.matricula, t.email, t.setor))
    t.id_tecnico = cur.lastrowid
    return cur.lastrowid

def buscar_tecnico(self, id_tecnico: int) -> Optional[Tecnico]:
    rows = self._consultar("SELECT * FROM tecnico WHERE id_tecnico = ?",
                           (id_tecnico,))
    return self._linha_para_tecnico(rows[0]) if rows else None

def buscar_tecnico_por_matricula(self, matricula: str) -> Optional[Tecnico]:
    rows = self._consultar("SELECT * FROM tecnico WHERE matricula = ?",
                           (matricula.strip().upper(),))
    return self._linha_para_tecnico(rows[0]) if rows else None

def listar_tecnicos(self) -> list[Tecnico]:
    return [self._linha_para_tecnico(r) for r in
            self._consultar("SELECT * FROM tecnico ORDER BY nome_tecnico")]

def atualizar_tecnico(self, t: Tecnico) -> bool:
    self._exigir_id(t.id_tecnico, "Tecnico")
    with self._transacao() as c:
        cur = c.execute(
            "UPDATE tecnico SET nome_tecnico = ?, matricula = ?, email = ?, "
            "setor = ? WHERE id_tecnico = ?",
            (t.nome_tecnico, t.matricula, t.email, t.setor, t.id_tecnico))
    return cur.rowcount > 0

def remover_tecnico(self, id_tecnico: int) -> bool:
    with self._transacao() as c:
        cur = c.execute("DELETE FROM tecnico WHERE id_tecnico = ?",
                        (id_tecnico,))
    return cur.rowcount > 0

# ============================================================================
# Equipamento
# ============================================================================
@staticmethod
def _linha_para_equipamento(r: sqlite3.Row) -> Equipamento:
    return Equipamento(r["codigo_tag"], r["nome_equipamento"],
                       r["fabricante"], r["modelo"],
                       id_equipamento=r["id_equipamento"])
```

```
def inserir_equipamento(self, e: Equipamento) -> int:
    with self._transacao() as c:
        cur = c.execute(
            "INSERT INTO equipamento (codigo_tag, nome_equipamento, "
            "fabricante, modelo) VALUES (?, ?, ?, ?)",
            (e.codigo_tag, e.nome_equipamento, e.fabricante, e.modelo))
    e.id_equipamento = cur.lastrowid
    return cur.lastrowid

def buscar_equipamento(self, id_equipamento: int) -> Optional[Equipamento]:
    rows = self._consultar(
        "SELECT * FROM equipamento WHERE id_equipamento = ?",
        (id_equipamento,))
    return self._linha_para_equipamento(rows[0]) if rows else None

def buscar_equipamento_por_tag(self, codigo_tag: str) -> Optional[Equipamento]:
    rows = self._consultar("SELECT * FROM equipamento WHERE codigo_tag = ?",
                           (codigo_tag.strip().upper(),))
    return self._linha_para_equipamento(rows[0]) if rows else None

def listar_equipamentos(self) -> list[Equipamento]:
    return [self._linha_para_equipamento(r) for r in
            self._consultar("SELECT * FROM equipamento ORDER BY codigo_tag")]

def atualizar_equipamento(self, e: Equipamento) -> bool:
    self._exigir_id(e.id_equipamento, "Equipamento")
    with self._transacao() as c:
        cur = c.execute(
            "UPDATE equipamento SET codigo_tag = ?, nome_equipamento = ?, "
            "fabricante = ?, modelo = ? WHERE id_equipamento = ?",
            (e.codigo_tag, e.nome_equipamento, e.fabricante, e.modelo,
             e.id_equipamento))
    return cur.rowcount > 0

def remover_equipamento(self, id_equipamento: int) -> bool:
    with self._transacao() as c:
        cur = c.execute("DELETE FROM equipamento WHERE id_equipamento = ?",
                        (id_equipamento,))
    return cur.rowcount > 0

# ============================================================================
# Medição
# ============================================================================
@staticmethod
def _linha_para_medicao(r: sqlite3.Row) -> Medicao:
    return Medicao(r["id_equipamento"], r["id_tecnico"],
                   r["parametro_medido"], r["valor_nominal"],
                   r["tolerancia_min"], r["tolerancia_max"],
                   r["valor_obtido"], id_medicao=r["id_medicao"],
                   data_medicao=r["data_medicao"])

def inserir_medicao(self, m: Medicao) -> int:
    # O status gravado vem da regra da classe, não de quem chama.
    with self._transacao() as c:
        cur = c.execute(
            "INSERT INTO medicao (id_equipamento, id_tecnico, "
            "parametro_medido, valor_nominal, tolerancia_min, "
            "tolerancia_max, valor_obtido, status_resultado) "
            "VALUES (?, ?, ?, ?, ?, ?, ?, ?)",
            (m.id_equipamento, m.id_tecnico, m.parametro_medido,
             m.valor_nominal, m.tolerancia_min, m.tolerancia_max,
             m.valor_obtido, m.status_resultado))
    m.id_medicao = cur.lastrowid
    m.data_medicao = self._consultar(
        "SELECT data_medicao FROM medicao WHERE id_medicao = ?",
        (cur.lastrowid,))[0]["data_medicao"]
    return cur.lastrowid

def buscar_medicao(self, id_medicao: int) -> Optional[Medicao]:
    rows = self._consultar("SELECT * FROM medicao WHERE id_medicao = ?",
                           (id_medicao,))
    return self._linha_para_medicao(rows[0]) if rows else None

def listar_medicoes(self) -> list[Medicao]:
```

```
    return [self._linha_para_medicao(r) for r in
            self._consultar("SELECT * FROM medicao ORDER BY id_medicao")]

def atualizar_medicao(self, m: Medicao) -> bool:
    """Regrava o ensaio com o status recalculado pelos novos valores."""
    self._exigir_id(m.id_medicao, "Medicao")
    with self._transacao() as c:
        cur = c.execute(
            "UPDATE medicao SET id_equipamento = ?, id_tecnico = ?, "
            "parametro_medido = ?, valor_nominal = ?, tolerancia_min = ?, "
            "tolerancia_max = ?, valor_obtido = ?, status_resultado = ? "
            "WHERE id_medicao = ?",
            (m.id_equipamento, m.id_tecnico, m.parametro_medido,
             m.valor_nominal, m.tolerancia_min, m.tolerancia_max,
             m.valor_obtido, m.status_resultado, m.id_medicao))
    return cur.rowcount > 0

def remover_medicao(self, id_medicao: int) -> bool:
    with self._transacao() as c:
        cur = c.execute("DELETE FROM medicao WHERE id_medicao = ?",
                        (id_medicao,))
    return cur.rowcount > 0

# ========================================================================
# Consultas relacionais (JOIN)
# ========================================================================
def relatorio_calibracao(self, status: Optional[str] = None,
                         codigo_tag: Optional[str] = None) -> list[dict]:
    """Relatório de calibração: medição, equipamento e técnico (INNER JOIN).

    Aceita filtros opcionais por status e por tag, sempre parametrizados."""
    sql = (
        "SELECT m.id_medicao, e.codigo_tag, e.nome_equipamento, "
        "       m.parametro_medido, m.valor_nominal, m.tolerancia_min, "
        "       m.tolerancia_max, m.valor_obtido, m.status_resultado, "
        "       t.nome_tecnico, t.matricula, m.data_medicao "
        "FROM medicao m "
        "INNER JOIN equipamento e ON e.id_equipamento = m.id_equipamento "
        "INNER JOIN tecnico t     ON t.id_tecnico     = m.id_tecnico "
        "WHERE 1 = 1 "
    )
    params: list = []
    if status is not None:
        status = status.strip().upper()
        if status not in (STATUS_APROVADO, STATUS_REPROVADO):
            raise ErroValidacao(
                f"status deve ser {STATUS_APROVADO} ou {STATUS_REPROVADO}.")
        sql += "AND m.status_resultado = ? "
        params.append(status)
    if codigo_tag is not None:
        sql += "AND e.codigo_tag = ? "
        params.append(codigo_tag.strip().upper())
    sql += "ORDER BY e.codigo_tag, m.id_medicao"
    return [dict(r) for r in self._consultar(sql, tuple(params))]

def resumo_por_equipamento(self) -> list[dict]:
    """Totais de aprovadas e reprovadas por equipamento (LEFT JOIN com
    GROUP BY). Equipamentos ainda sem medição aparecem com zero."""
    sql = (
        "SELECT e.codigo_tag, e.nome_equipamento, "
        "       COUNT(m.id_medicao) AS total, "
        "       COALESCE(SUM(m.status_resultado = 'APROVADO'), 0)  AS aprovadas, "
        "       COALESCE(SUM(m.status_resultado = 'REPROVADO'), 0) AS reprovadas "
        "FROM equipamento e "
        "LEFT JOIN medicao m ON m.id_equipamento = e.id_equipamento "
        "GROUP BY e.id_equipamento ORDER BY e.codigo_tag"
    )
    return [dict(r) for r in self._consultar(sql)]

@staticmethod
def _exigir_id(valor, entidade: str) -> None:
    if valor is None:
        raise ErroValidacao(
            f"{entidade} ainda não foi gravado (sem id). Use inserir_* antes.")
```

Anexo B: código-fonte de 05_main.py

```
# ============================================================================
# UNIDADE CURRICULAR: PROJETO APLICADO I | SCHULZ S.A.
# AV 03 (MVP Parte 1) | Arquivo 05: script principal de demonstração
#
# Para executar:  python 05_main.py
#
# Roteiro:
#   1. Cria o banco e o esquema          4. UPDATE (correção de uma leitura)
#   2. Instancia os objetos de domínio   5. Tratamento de erros
#   3. Insere registros de teste         6. Consultas com JOIN e relatório
#
# Todos os nomes e valores abaixo são dados de teste fictícios.
# ============================================================================
import importlib
import os
import sys

# O arquivo se chama '04_classes.py' e começa com dígito, então o 'import'
# comum não funciona. O importlib aceita qualquer nome de módulo.
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
dominio = importlib.import_module("04_classes")

BancoDados = dominio.BancoDados
Tecnico = dominio.Tecnico
Equipamento = dominio.Equipamento
Medicao = dominio.Medicao
ErroValidacao = dominio.ErroValidacao
ErroPersistencia = dominio.ErroPersistencia

CAMINHO_BANCO = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                             "mvp_schulz_calibracao.db")


def titulo(texto: str) -> None:
    print("\n" + "=" * 100)
    print(texto)
    print("=" * 100)


def cadastrar_dados_de_teste(bd: BancoDados) -> list:
    titulo("2 e 3. INSTANCIAÇÃO DOS OBJETOS E INSERÇÃO DE REGISTROS DE TESTE")

    tecnicos = [
        Tecnico("Carlos Eduardo Pereira", "MT1001", "carlos.pereira@teste.com.br",
                "Metrologia"),
        Tecnico("Ana Beatriz Souza", "MT1002", "ana.souza@teste.com.br",
                "Metrologia"),
        Tecnico("Rafael Lima Costa", "US2001", "rafael.costa@teste.com.br",
                "Usinagem"),
    ]
    for t in tecnicos:
        bd.inserir_tecnico(t)
        print(f"  [OK] {t}")

    equipamentos = [
        Equipamento("703.1208", "Dispositivo de Controle", "CDM", "DC-120"),
        Equipamento("703.1315", "Dispositivo de Controle", "Usifer", "UF-45"),
        Equipamento("500.011", "Padrão Anel Liso", "Mitutoyo", "177-290"),
        Equipamento("810.0042", "Paquímetro Digital", "Mitutoyo", "500-196"),  # sem medição
    ]
    for e in equipamentos:
        bd.inserir_equipamento(e)
        print(f"  [OK] {e}")

    carlos, ana, rafael = tecnicos
    disp1, disp2, anel, _paq = equipamentos

    # No construtor, tolerancia_min e tolerancia_max já são os limites absolutos.
    # Na fábrica a_partir_de_desvios, informa-se o nominal e os desvios -/+.
    medicoes = [
        Medicao(disp1.id_equipamento, carlos.id_tecnico, "Diâmetro",
```

```
                25.000, 24.950, 25.030, 25.012),                    # aprovada
            Medicao(disp1.id_equipamento, carlos.id_tecnico, "Planicidade",
                0.000, 0.000, 0.020, 0.035),                        # reprovada (acima)
            Medicao.a_partir_de_desvios(
                disp1.id_equipamento, ana.id_tecnico, "Posição",
                50.000, -0.050, +0.050, 49.949),                    # reprovada (abaixo)
            Medicao.a_partir_de_desvios(
                disp2.id_equipamento, ana.id_tecnico, "Diâmetro",
                10.000, -0.020, +0.030, "10,030"),                  # no limite, aprovada
            Medicao(disp2.id_equipamento, rafael.id_tecnico, "Altura",
                75.500, 75.400, 75.600, 75.512),                    # aprovada
            Medicao(anel.id_equipamento, ana.id_tecnico, "Diâmetro Interno",
                40.000, 39.995, 40.005, 40.002),                    # aprovada
            Medicao(anel.id_equipamento, carlos.id_tecnico, "Diâmetro Interno",
                40.000, 39.995, 40.005, 40.011),                    # reprovada
        ]
        for m in medicoes:
            bd.inserir_medicao(m)
            print(f"  [OK] {m}  desvio={m.desvio:+.3f}")

        return medicoes


    def demonstrar_update(bd: BancoDados, medicao: Medicao) -> None:
        titulo("4. UPDATE: CORREÇÃO DE LEITURA (o status é recalculado automaticamente)")
        print(f"  Antes : id={medicao.id_medicao} obtido={medicao.valor_obtido} "
              f"-> {medicao.status_resultado}")
        medicao.valor_obtido = 0.015  # a leitura anterior (0.035) foi digitada errada
        bd.atualizar_medicao(medicao)
        recarregada = bd.buscar_medicao(medicao.id_medicao)
        print(f"  Depois: id={recarregada.id_medicao} obtido={recarregada.valor_obtido} "
              f"-> {recarregada.status_resultado} (lido de volta do banco)")

        tecnico = bd.buscar_tecnico_por_matricula("US2001")
        tecnico.setor = "Metrologia"
        bd.atualizar_tecnico(tecnico)
        print(f"  Técnico remanejado: {bd.buscar_tecnico(tecnico.id_tecnico)}")


    def demonstrar_tratamento_de_erros(bd: BancoDados, id_tecnico: int,
                                       id_equipamento: int) -> None:
        titulo("5. TRATAMENTO DE ERROS (todos os casos abaixo devem ser bloqueados)")

        def tentar(descricao, acao):
            try:
                acao()
                print(f"  [FALHA] '{descricao}' NÃO foi bloqueado!")
            except ErroValidacao as e:
                print(f"  [BLOQUEADO na validação do domínio] {descricao}\n"
                      f"       {e}")
            except ErroPersistencia as e:
                print(f"  [BLOQUEADO pelo SQLite] {descricao}\n       {e}")

        tentar("E-mail sem formato válido",
               lambda: Tecnico("Fulano de Tal", "MT9999", "email-invalido", "Metrologia"))
        tentar("Valor obtido não numérico",
               lambda: Medicao(id_equipamento, id_tecnico, "Diâmetro", 10, 9.9, 10.1, "abc"))
        tentar("Limite inferior maior que o superior",
               lambda: Medicao(id_equipamento, id_tecnico, "Diâmetro", 10, 10.2, 10.1, 10))
        tentar("Nominal fora da faixa de tolerância",
               lambda: Medicao(id_equipamento, id_tecnico, "Diâmetro", 10, 10.1, 10.2, 10.15))
        tentar("Matrícula duplicada (UNIQUE)",
               lambda: bd.inserir_tecnico(
                   Tecnico("Outro Nome", "MT1001", "outro@teste.com.br", "Metrologia")))
        tentar("Tag duplicada (UNIQUE)",
               lambda: bd.inserir_equipamento(
                   Equipamento("703.1208", "Qualquer", "CDM", "X1")))
        tentar("Medição com equipamento inexistente (FOREIGN KEY)",
               lambda: bd.inserir_medicao(
                   Medicao(9999, id_tecnico, "Diâmetro", 10, 9.9, 10.1, 10.0)))
        tentar("Excluir técnico que possui medições (ON DELETE RESTRICT)",
               lambda: bd.remover_tecnico(id_tecnico))
        tentar("Filtro de status inválido no relatório",
               lambda: bd.relatorio_calibracao(status="TALVEZ"))
```

```
def imprimir_relatorio(linhas: list, titulo_relatorio: str) -> None:
    titulo(titulo_relatorio)
    if not linhas:
        print("  (nenhum registro)")
        return
    cab = (f"{'ID':>3} | {'TAG':<9} | {'PARÂMETRO':<17} | {'NOMINAL':>8} | "
           f"{'LIM.MIN':>8} | {'LIM.MAX':>8} | {'OBTIDO':>8} | {'STATUS':<9} | "
           f"{'TÉCNICO':<22}")
    print(cab)
    print("-" * len(cab))
    for r in linhas:
        print(f"{r['id_medicao']:>3} | {r['codigo_tag']:<9} | "
              f"{r['parametro_medido']:<17} | {r['valor_nominal']:>8.3f} | "
              f"{r['tolerancia_min']:>8.3f} | {r['tolerancia_max']:>8.3f} | "
              f"{r['valor_obtido']:>8.3f} | {r['status_resultado']:<9} | "
              f"{r['nome_tecnico']:<22}")


def main() -> None:
    titulo("MVP PARTE 1: SISTEMA DE CALIBRAÇÃO | SCHULZ S.A. (dados de teste)")

    # O banco é recriado a cada execução para o resultado ser reproduzível.
    if os.path.exists(CAMINHO_BANCO):
        os.remove(CAMINHO_BANCO)

    try:
        with BancoDados(CAMINHO_BANCO) as bd:
            titulo("1. CONEXÃO E CRIAÇÃO DO ESQUEMA (tecnico, equipamento, medicao)")
            bd.criar_esquema()
            print(f"  Banco criado: {os.path.basename(CAMINHO_BANCO)}")

            medicoes = cadastrar_dados_de_teste(bd)
            demonstrar_update(bd, medicoes[1])  # planicidade: 0.035 para 0.015
            demonstrar_tratamento_de_erros(
                bd, medicoes[0].id_tecnico, medicoes[0].id_equipamento)

            imprimir_relatorio(bd.relatorio_calibracao(),
                               "6.1 RELATÓRIO DE CALIBRAÇÃO COMPLETO "
                               "(medicao JOIN equipamento JOIN tecnico)")
            imprimir_relatorio(bd.relatorio_calibracao(status="REPROVADO"),
                               "6.2 SOMENTE MEDIÇÕES REPROVADAS")
            imprimir_relatorio(bd.relatorio_calibracao(codigo_tag="703.1208"),
                               "6.3 HISTÓRICO DO EQUIPAMENTO 703.1208")

            titulo("6.4 RESUMO POR EQUIPAMENTO (LEFT JOIN com GROUP BY)")
            print(f"{'TAG':<10} | {'EQUIPAMENTO':<24} | {'TOTAL':>5} | "
                  f"{'APROV.':>6} | {'REPROV.':>7}")
            print("-" * 64)
            for r in bd.resumo_por_equipamento():
                print(f"{r['codigo_tag']:<10} | {r['nome_equipamento']:<24} | "
                      f"{r['total']:>5} | {r['aprovadas']:>6} | {r['reprovadas']:>7}")

        print("\nExecução concluída com sucesso.")
    except (ErroPersistencia, ErroValidacao) as e:
        print(f"\nERRO FATAL: {e}")
        sys.exit(1)


if __name__ == "__main__":
    main()
```

Anexo C: saída real da execução de 05_main.py

```
================================================================================================
MVP PARTE 1: SISTEMA DE CALIBRAÇÃO | SCHULZ S.A. (dados de teste)
================================================================================================

================================================================================================
1. CONEXÃO E CRIAÇÃO DO ESQUEMA (tecnico, equipamento, medicao)
================================================================================================
  Banco criado: mvp_schulz_calibracao.db

================================================================================================
2 e 3. INSTANCIAÇÃO DOS OBJETOS E INSERÇÃO DE REGISTROS DE TESTE
================================================================================================
  [OK] Tecnico(id=1, matricula='MT1001', nome='Carlos Eduardo Pereira', setor='Metrologia')
  [OK] Tecnico(id=2, matricula='MT1002', nome='Ana Beatriz Souza', setor='Metrologia')
  [OK] Tecnico(id=3, matricula='US2001', nome='Rafael Lima Costa', setor='Usinagem')
  [OK] Equipamento(id=1, tag='703.1208', nome='Dispositivo de Controle')
  [OK] Equipamento(id=2, tag='703.1315', nome='Dispositivo de Controle')
  [OK] Equipamento(id=3, tag='500.011', nome='Padrão Anel Liso')
  [OK] Equipamento(id=4, tag='810.0042', nome='Paquímetro Digital')
  [OK] Medicao(id=1, eq=1, tec=1, 'Diâmetro', obtido=25.012, APROVADO)  desvio=+0.012
  [OK] Medicao(id=2, eq=1, tec=1, 'Planicidade', obtido=0.035, REPROVADO)  desvio=+0.035
  [OK] Medicao(id=3, eq=1, tec=2, 'Posição', obtido=49.949, REPROVADO)  desvio=-0.051
  [OK] Medicao(id=4, eq=2, tec=2, 'Diâmetro', obtido=10.03, APROVADO)  desvio=+0.030
  [OK] Medicao(id=5, eq=2, tec=3, 'Altura', obtido=75.512, APROVADO)  desvio=+0.012
  [OK] Medicao(id=6, eq=3, tec=2, 'Diâmetro Interno', obtido=40.002, APROVADO)  desvio=+0.002
  [OK] Medicao(id=7, eq=3, tec=1, 'Diâmetro Interno', obtido=40.011, REPROVADO)  desvio=+0.011

================================================================================================
4. UPDATE: CORREÇÃO DE LEITURA (o status é recalculado automaticamente)
================================================================================================
  Antes : id=2 obtido=0.035 -> REPROVADO
  Depois: id=2 obtido=0.015 -> APROVADO (lido de volta do banco)
  Técnico remanejado: Tecnico(id=3, matricula='US2001', nome='Rafael Lima Costa', setor='Metrologia')

================================================================================================
5. TRATAMENTO DE ERROS (todos os casos abaixo devem ser bloqueados)
================================================================================================
  [BLOQUEADO na validação do domínio] E-mail sem formato válido
       'email' inválido: 'email-invalido'.
  [BLOQUEADO na validação do domínio] Valor obtido não numérico
       'valor_obtido' deve ser numérico (recebido: 'abc').
  [BLOQUEADO na validação do domínio] Limite inferior maior que o superior
       tolerancia_min não pode ser maior que tolerancia_max.
  [BLOQUEADO na validação do domínio] Nominal fora da faixa de tolerância
       valor_nominal deve estar dentro da faixa [tolerancia_min, tolerancia_max].
  [BLOQUEADO pelo SQLite] Matrícula duplicada (UNIQUE)
       Registro duplicado: já existe um registro com este 'matricula'.
  [BLOQUEADO pelo SQLite] Tag duplicada (UNIQUE)
       Registro duplicado: já existe um registro com este 'codigo_tag'.
  [BLOQUEADO pelo SQLite] Medição com equipamento inexistente (FOREIGN KEY)
       Violação de integridade referencial: o registro referenciado não existe ou ainda possui medições vinculadas.
  [BLOQUEADO pelo SQLite] Excluir técnico que possui medições (ON DELETE RESTRICT)
       Violação de integridade referencial: o registro referenciado não existe ou ainda possui medições vinculadas.
  [BLOQUEADO na validação do domínio] Filtro de status inválido no relatório
       status deve ser APROVADO ou REPROVADO.

================================================================================================
6.1 RELATÓRIO DE CALIBRAÇÃO COMPLETO (medicao JOIN equipamento JOIN tecnico)
================================================================================================
 ID | TAG       | PARÂMETRO         |  NOMINAL |  LIM.MIN |  LIM.MAX |   OBTIDO | STATUS    | TÉCNICO
------------------------------------------------------------------------------------------------------------------------
  6 | 500.011   | Diâmetro Interno  |   40.000 |   39.995 |   40.005 |   40.002 | APROVADO  | Ana Beatriz Souza
  7 | 500.011   | Diâmetro Interno  |   40.000 |   39.995 |   40.005 |   40.011 | REPROVADO | Carlos Eduardo Pereira
  1 | 703.1208  | Diâmetro          |   25.000 |   24.950 |   25.030 |   25.012 | APROVADO  | Carlos Eduardo Pereira
  2 | 703.1208  | Planicidade       |    0.000 |    0.000 |    0.020 |    0.015 | APROVADO  | Carlos Eduardo Pereira
  3 | 703.1208  | Posição           |   50.000 |   49.950 |   50.050 |   49.949 | REPROVADO | Ana Beatriz Souza
  4 | 703.1315  | Diâmetro          |   10.000 |    9.980 |   10.030 |   10.030 | APROVADO  | Ana Beatriz Souza
  5 | 703.1315  | Altura            |   75.500 |   75.400 |   75.600 |   75.512 | APROVADO  | Rafael Lima Costa

================================================================================================
6.2 SOMENTE MEDIÇÕES REPROVADAS
================================================================================================
 ID | TAG       | PARÂMETRO         |  NOMINAL |  LIM.MIN |  LIM.MAX |   OBTIDO | STATUS    | TÉCNICO
------------------------------------------------------------------------------------------------------------------------
  7 | 500.011   | Diâmetro Interno  |   40.000 |   39.995 |   40.005 |   40.011 | REPROVADO | Carlos Eduardo Pereira
  3 | 703.1208  | Posição           |   50.000 |   49.950 |   50.050 |   49.949 | REPROVADO | Ana Beatriz Souza

================================================================================================
6.3 HISTÓRICO DO EQUIPAMENTO 703.1208
================================================================================================
 ID | TAG       | PARÂMETRO         |  NOMINAL |  LIM.MIN |  LIM.MAX |   OBTIDO | STATUS    | TÉCNICO
------------------------------------------------------------------------------------------------------------------------
  1 | 703.1208  | Diâmetro          |   25.000 |   24.950 |   25.030 |   25.012 | APROVADO  | Carlos Eduardo Pereira
  2 | 703.1208  | Planicidade       |    0.000 |    0.000 |    0.020 |    0.015 | APROVADO  | Carlos Eduardo Pereira
  3 | 703.1208  | Posição           |   50.000 |   49.950 |   50.050 |   49.949 | REPROVADO | Ana Beatriz Souza
```

```
================================================================================================================
6.4 RESUMO POR EQUIPAMENTO (LEFT JOIN com GROUP BY)
================================================================================================================
TAG         | EQUIPAMENTO               | TOTAL | APROV. | REPROV.
----------------------------------------------------------------
500.011     | Padrão Anel Liso          |     2 |      1 |       1
703.1208    | Dispositivo de Controle   |     3 |      2 |       1
703.1315    | Dispositivo de Controle   |     2 |      2 |       0
810.0042    | Paquímetro Digital        |     0 |      0 |       0

Execução concluída com sucesso.
```